

template

December 7, 2025

```
[1]: import numpy as np
      from numpy.linalg import solve, norm
```

Implementation of the forward Euler, backward Euler, and trapezoidal methods For the implicit method we need a Newton method solver:

```
[2]: def newton(f,Df,x0,n_iter,tol):
      x = x0.copy()
      for i in range(n_iter):
          fsol = f(x)
          if fsol.dot(fsol) < tol**2:
              break
          delta = solve(Df(x),fsol)
          x -= delta
      assert i < n_iter-1
      return x

      # Forward Euler stepper
      def forwardEuler(f,tau,t0,y0):
          return y0 + tau*f(t0,y0)
      # Backward Euler stepper
      def backwardEuler(f,Df,tau,t0,y0):
          F = lambda x: x - y0 - tau*f(t0+tau,x)
          DF = lambda x: np.eye(len(y0)) - tau*Df(t0+tau,x)
          return newton(F,DF,y0,100,1e-8)
      # Trapezoidal stepper
      def trapezoidal(f,Df,tau,t0,y0):
          y = y0 + tau/2*f(t0,y0)
          F = lambda x: x - y - tau/2*f(t0+tau,x)
          DF = lambda x: np.eye(len(y0)) - tau/2*Df(t0+tau,x)
          return newton(F,DF,y0,100,1e-8)
```

This function evolves the solution from 0 to $N\tau$ and returns $(t_n)_n$ and $(y_{i,n})_n$ for $n = 0, \dots, N-1$ and $i = 0, \dots, r-1$ where r is the size of the ODE system. For the heat equation r will be the product of the number of components in the heat equation (q) and the number of points (P) chosen for the spatial discretization:

```
[3]: def evolve(stepper,tau,N,y0):
    t = np.zeros( N+1 )
    y = np.zeros( (len(y0), N+1) )
    t[0] = 0
    y[:,0] = y0
    for n in range(N):
        t[n+1] = t[n] + tau
        y[:,n+1] = stepper(tau,t[n],y[:,n])
    return t,y
```

Setup for a simple ODE with two components and a vector valued heat equation:

```
[4]: class VectorProblem:
    def __init__(self):
        self.T = 10
        self.fcount = 0
    def NO(self,explicit=True):
        # this is just some value that lies well above the stability limit FE
        return self.T * 20 if explicit else self.T * 5

    def exact(self,t):
        return np.array([ 2*np.exp(t)/(2*np.exp(t)-1),
                          (-2*np.exp(t))/(4*np.exp(2*t)-4*np.exp(t)+1) ])

    def u0(self):
        return self.exact(0)
    def f(self,t,y):
        self.fcount += 1
        return np.array([y[1], y[1]*(1-2*y[0])])
    def Df(self,t,y):
        return np.array([[0,1], [-2*y[1], 1-2*y[0]]])

# vector valued heat equation
class Heat:
    def __init__(self,q,points):
        self.points = points-2 # without two end points
        self.x = np.linspace(0,1,points)[1:-1]
        # parameters
        self.mu = 1/50
        self.h = 1/(points-1)
        self.q = q # system of q uncoupled heat equations
        self.T = 1 # final time
        self.uL = np.array(q*[1.]) # left side boundary conditions
        self.uR = np.array(q*[1.]) # right side boundary conditions
        h = 1/points # step size for space
        self.fcount = 0

    def NO(self,explicit=True):
        # this is the expected limit for stability of the FE method
```

```

        return ( 2 * int(self.T * self.mu / self.h**2 ) if explicit else 5 )

def exact(self,t):
    mode = lambda v: np.exp(-(v*np.pi)**2*self.mu*t)*\
        np.sin(v*np.pi*self.x)
    ret = np.array(self.q*self.points*[1.])
    # add something to the first component
    ret[:,self.q] += mode(2)
    # also add something to the second component (if it exists)
    if self.q > 1:
        ret[1::self.q] += mode(3) + mode(4)
    return ret
# initial conditions
def u0(self):
    return self.exact(0)

# matrix for diffusion operator (d_xx)
def diffusion(self):
    N = self.q * self.points
    ret = np.zeros((N,N))
    for i in range(self.points):
        for j in range(self.q):
            if i>0:
                ret[self.q*i+j][self.q*(i-1)+j] = self.mu/self.h**2
            ret[self.q*i+j][self.q*i+j] = -2*self.mu/self.h**2
            if i<self.points-1:
                ret[self.q*i+j][self.q*(i+1)+j] = self.mu/self.h**2
    return ret

# right hand side for ODE
def f(self,t,u):
    self.fcount += 1
    ret = self.diffusion()@u
    # boundary conditions
    for j in range(self.q):
        ret[j] += self.mu/self.h**2*self.uL[j]
        ret[-self.q+j] += self.mu/self.h**2*self.uR[j]
    return ret
# and its Jacobian
def Df(self,t,u):
    ret = self.diffusion()
    return ret

```

Solve a given problem with a sequence of τ :

```

[5]: def compute(stepper,name,verbose=False):
    maxErrs = []

```

```

# solve problem with  $N=N0*2**i$  ( $i=0,1,2,3,4$ )
for factor in [2**i for i in range(5)]:
    # for explicit we need to take stability restrictions into account
    # while for the implicit methods we chose  $N0=5$ 
    N = factor * prob.N0(explicit=(stepper=="fe"))
    tau = prob.T/N
    y0 = prob.u0().flatten('F')
    prob.fcount = 0
    t,y = evolve(stepper=stepper, tau=tau, N=N, y0=y0)
    # compute the maximum error (if available)
    if hasattr(prob, "exact"):
        maxErr = 0
        for n in range(len(t)):
            maxErr = max(maxErr, norm(y[:,n]-prob.exact(t[n])))
    else:
        maxErr = None
    maxErrs.append(maxErr)
    if verbose:
        print(name, N, prob.fcount, maxErr)
    # we only do one simulation with the explicit methods - for the
    # heat equation that is expensive enough as it is
    if stepper == "fe":
        break
return maxErrs

```

Main part: iterate over available solvers and store the errors

```

[6]: prob = VectorProblem()
# prob = Heat(2,600)

fe = lambda tau,t0,y0: forwardEuler(prob.f, tau,t0,y0)
be = lambda tau,t0,y0: backwardEuler(prob.f, prob.Df, tau,t0,y0)
tr = lambda tau,t0,y0: trapezoidal(prob.f, prob.Df, tau,t0,y0)

# loop over steppers
maxErrs = {}
for stepper,name in zip([fe,be,tr], ["FE","BE","TR"]):
    maxErrs[name] = compute(stepper,name,verbose=True)

```

```

FE 200 200 0.06525496682304095
BE 50 158 0.19494367091283835
BE 100 272 0.10580202782876653
BE 200 496 0.05635231918047668
BE 400 940 0.029146222394353164
BE 800 1797 0.014831273590207777
TR 50 184 0.04250295887078444
TR 100 353 0.010133128708409125
TR 200 681 0.002522719684448781

```

```
TR 400 1322 0.000628761497644405
TR 800 2570 0.00015716572597141204
```

```
[7]: maxErrs
```

```
[7]: {'FE': [np.float64(0.06525496682304095)],
      'BE': [np.float64(0.19494367091283835),
              np.float64(0.10580202782876653),
              np.float64(0.05635231918047668),
              np.float64(0.029146222394353164),
              np.float64(0.014831273590207777)],
      'TR': [np.float64(0.04250295887078444),
              np.float64(0.010133128708409125),
              np.float64(0.002522719684448781),
              np.float64(0.000628761497644405),
              np.float64(0.00015716572597141204)]}
```

```
[ ]:
```